

CoreSys Kernel Architecture

The CoreSys kernel is a freestanding operating system kernel written in **Kernel C (KC)**. Unlike hosted C environments, the kernel does not rely on the C standard library. Every subsystem required by the operating system—including memory management, drivers, scheduling, synchronization, and system services—is implemented directly by CoreSys.

The kernel is responsible for initializing hardware, creating the runtime environment, exposing system services, managing tasks, and eventually shutting the system down. Once the kernel has been deinitialized, execution enters **Restricted Kernel C (RKC)**, where only the most fundamental machine-level functionality remains available.

Drivers

CoreSys drivers provide direct access to hardware devices and low-level platform services. Drivers are considered part of the kernel and execute in **Kernel C**.

Examples of drivers include:

- ATA storage driver
- AHCI driver
- NVMe driver
- ACPI driver
- COPFS filesystem driver
- Display/terminal driver
- Keyboard driver
- Memory manager
- Interrupt controller
- Task scheduler

Drivers are designed to communicate directly with hardware and other kernel subsystems. Because of this, they are expected to avoid unnecessary dependencies and remain as self-contained as possible.

Driver Restrictions

Kernel drivers **must not** use the `CS_CORE` interface.

`CS_CORE` is intended for higher-level kernel components and applications that interact with the CoreSys API. Drivers instead operate using internal kernel functions, hardware registers, memory-mapped I/O, I/O ports, interrupts, synchronization primitives, and other low-level facilities.

Drivers may:

- Read and write hardware registers.
- Perform DMA operations.
- Allocate kernel memory.

- Register interrupts.
- Expose services to other kernel components.
- Interact with other drivers through kernel interfaces.

Drivers should not:

- Depend on `CS_CORE`.
- Assume application-level abstractions exist.
- Use interfaces that are unavailable after kernel deinitialization.

Keeping drivers independent from `CS_CORE` allows them to remain portable within the kernel and simplifies early boot and shutdown stages.

kmain()

`kmain()` is the primary entry point of the CoreSys kernel.

After the bootloader transfers control to the kernel, execution begins in `kmain()`. This function is responsible for bringing the operating system from an uninitialized state into a fully operational kernel environment.

Typical responsibilities include:

- Verifying boot information.
- Initializing CPU-specific features.
- Initializing memory management.
- Setting up interrupt handling.
- Initializing timers.
- Starting hardware drivers.
- Detecting storage devices.
- Mounting or initializing filesystems.
- Initializing the task scheduler.
- Creating kernel tasks.
- Starting system services.
- Preparing the system API.
- Launching the initial task or program.

Once initialization is complete, `kmain()` transitions control to the scheduler or primary execution loop.

Kernel Runtime

During normal operation, the kernel provides numerous services, including:

- Task scheduling
- Memory allocation
- Device management
- Filesystem access

- System calls
- Driver communication
- Synchronization primitives
- Logging and debugging
- Hardware abstraction

These services collectively define the **Kernel C** environment.

Kernel components, drivers, and kernel tasks may freely use these facilities while the kernel remains initialized.

Kernel Shutdown

Eventually the kernel may be instructed to shut down.

This can occur because of:

- A system halt request.
- A reboot request.
- A fatal kernel error.
- A controlled shutdown.
- A system deinitialization request.

Before entering the final shutdown stage, the kernel performs an orderly cleanup of its resources.

Typical shutdown operations include:

- Stopping task scheduling.
- Terminating or suspending tasks.
- Flushing pending filesystem operations.
- Releasing allocated kernel resources.
- Deinitializing hardware drivers.
- Disabling interrupts where appropriate.
- Shutting down kernel subsystems.
- Destroying kernel runtime services.

After these operations complete, the kernel runtime no longer exists.

RKC (Restricted Kernel C)

After `cs_deinit()` and the kernel deinitialization sequence complete, execution enters **Restricted Kernel C (RKC)**.

RKC represents the lowest software execution environment provided by CoreSys before complete processor shutdown.

Unlike Kernel C, RKC no longer has access to the kernel runtime.

This means there are **no kernel services available**.

The following functionality is no longer guaranteed to exist:

- `CS_CORE`
- `core.*`
- `cs_task`
- Kernel allocators
- Filesystem interfaces
- Driver APIs
- Scheduler services
- System service libraries
- Kernel logging
- High-level synchronization primitives
- Standard kernel helper functions

Only functionality that still exists after kernel deinitialization may be used.

Typically this consists of:

- CPU instructions
- Inline assembly
- Direct memory access
- Memory-mapped I/O
- Port I/O
- CPU registers
- Hardware registers
- Platform firmware interfaces (when still valid)

Because nearly all kernel infrastructure has been removed, software executing in RKC is responsible for managing every remaining operation directly.

Purpose of RKC

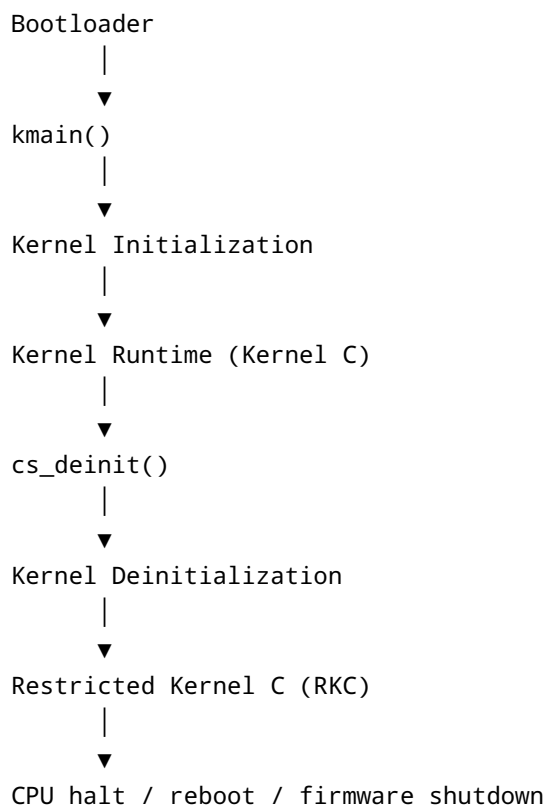
Restricted Kernel C exists to perform the final machine-level operations after the operating system has shut itself down.

Examples include:

- Executing the final processor halt.
- Performing architecture-specific shutdown operations.
- Triggering hardware reset mechanisms.
- Writing directly to firmware registers.
- Sending ACPI shutdown commands.
- Executing final assembly routines.
- Performing emergency recovery code.
- Entering infinite halt loops.

A typical RKC halt sequence might eventually end with the CPU `hlt` instruction after interrupts have been disabled.

Transition Between KC and RKC



RKC is intentionally minimal. It represents the final execution stage of CoreSys after the operating system has removed its own runtime environment, leaving only direct interaction with the processor and hardware.